AI-Powered Food & Cosmetic Product Safety Assessment System — Master Plan

Project Codename: **SafeScan**

1. Creative File Structure Proposal

1.1 — Expo Mobile App (`/mobile`)

```
mobile/
├── app/                          # Expo Router (file-based routing)
│   ├── _layout.tsx               # Root layout — global providers, fonts, theme
│   ├── index.tsx                 # Landing / splash → redirects to (tabs)
│   ├── (auth)/                   # Auth group (unauthenticated)
│   │   ├── _layout.tsx
│   │   ├── welcome.tsx           # Onboarding carousel
│   │   ├── login.tsx
│   │   └── register.tsx
│   ├── (tabs)/                   # Main tabbed experience (authenticated)
│   │   ├── _layout.tsx           # Tab bar configuration
│   │   ├── scan.tsx              # Camera capture + upload screen
│   │   ├── history.tsx           # Past scan reports
│   │   ├── compare.tsx           # Side-by-side product comparison
│   │   └── profile.tsx           # User prefs, allergies, dietary profile
│   └── report/
│       └── [id].tsx              # Dynamic route — full report view
│
├── components/                   # Reusable UI components
│   ├── ui/                       # Atomic design system
│   │   ├── Button.tsx
│   │   ├── Badge.tsx             # Assessment badges (Favorable, Caution, etc.)
│   │   ├── Card.tsx
│   │   ├── ConfidenceMeter.tsx   # Animated confidence arc
│   │   ├── GlassPanel.tsx        # Glassmorphism container
│   │   ├── GradientText.tsx
│   │   ├── IconPill.tsx          # Tag-style ingredient chips
│   │   ├── ProgressRing.tsx
│   │   └── Skeleton.tsx          # Loading skeletons
│   ├── scan/                     # Scan-flow specific
│   │   ├── CameraOverlay.tsx     # Viewfinder with label-region guide
│   │   ├── ImagePreview.tsx
│   │   ├── QualityIndicator.tsx  # Real-time image quality feedback
│   │   └── ScanProgress.tsx      # Multi-step processing animation
│   ├── report/                   # Report rendering
│   │   ├── AssessmentHeader.tsx  # Product name + overall badge
│   │   ├── BenefitsSection.tsx   # Green findings
│   │   ├── ConcernsSection.tsx   # Amber/red findings
│   │   ├── IngredientList.tsx    # Full ingredient breakdown
```

```
│   │   ├── IngredientRow.tsx     # Single ingredient with status icon
│   │   ├── NutritionSummary.tsx  # Food-specific nutrition panel
│   │   ├── SourceReference.tsx   # Evidence source pill
│   │   └── DisclaimerBanner.tsx
│   └── profile/
│       ├── AllergySelector.tsx
│       ├── DietaryToggle.tsx
│       └── JurisdictionPicker.tsx
│
├── services/                     # Appwrite SDK service layer
│   ├── appwrite.ts               # Client singleton + exports
│   ├── auth.service.ts           # Login, register, session management
│   ├── scan.service.ts           # Upload image → trigger function → poll result
│   ├── report.service.ts         # Fetch, list, delete reports
│   ├── profile.service.ts        # User preferences CRUD
│   └── storage.service.ts        # Image upload/download helpers
│
├── stores/                       # State management (Zustand)
│   ├── useAuthStore.ts
│   ├── useScanStore.ts           # Current scan state machine
│   ├── useReportStore.ts
│   └── useProfileStore.ts
│
├── hooks/                        # Custom React hooks
│   ├── useCamera.ts
│   ├── useImagePicker.ts
│   ├── useAppwrite.ts            # Generic Appwrite query hook
│   └── useDebounce.ts
│
├── constants/                    # App-wide constants
│   ├── theme.ts                  # Colors, spacing, typography, gradients
│   ├── allergens.ts              # Allergen display names + icons
│   ├── categories.ts             # Product category enum + metadata
│   └── assessments.ts            # Assessment tier definitions
│
├── utils/                        # Pure utility functions
│   ├── formatters.ts             # Date, percentage, confidence formatting
│   ├── validators.ts
│   └── imageHelpers.ts           # Resize, compress before upload
│
├── assets/                       # Static assets
│   ├── fonts/
│   ├── images/
│   ├── icons/
│   └── animations/               # Lottie files for scan progress, etc.
│
├── app.json                      # Expo config
├── package.json
└── tsconfig.json
```

1.2 — Appwrite Functions (`/functions`)

```
functions/
├── assess-product/              # THE core function
│   ├── src/
│   │   ├── main.js              # Entry point — router
│   │   ├── ocr/
│   │   │   └── extractText.js   # Gemini vision: image → raw text
│   │   ├── classification/
│   │   │   └── classifyProduct.js # Gemini: raw text → product category
│   │   ├── extraction/
│   │   │   └── extractIngredients.js # Gemini: raw text → structured ingredients
│   │   ├── normalization/
│   │   │   └── normalizeIngredients.js # Gemini + DB: map to canonical names
│   │   ├── safety/
│   │   │   ├── lookupIngredients.js   # Query Appwrite DB for regulatory data
│   │   │   ├── foodRules.js           # Deterministic food safety rules
│   │   │   ├── cosmeticRules.js       # Deterministic cosmetic safety rules
│   │   │   └── nutritionRules.js      # Nutrition threshold analysis
│   │   ├── assessment/
│   │   │   ├── scoreEngine.js         # Aggregate findings → overall verdict
│   │   │   └── explanationGenerator.js # Gemini: structured data → plain English
│   │   ├── providers/                      #  🔌 Multi-AI provider abstraction
│   │   │   ├── aiProvider.js               # Common interface / base class
│   │   │   ├── geminiProvider.js           # Google Gemini 2.0 Flash
│   │   │   ├── openaiProvider.js           # OpenAI GPT-4o (future)
│   │   │   ├── claudeProvider.js           # Anthropic Claude (future)
│   │   │   └── providerFactory.js          # Reads AI_PROVIDER env → returns
instance
│   │   └── utils/
│   │       ├── appwriteClient.js      # node-appwrite singleton
│   │       └── confidenceCalc.js      # OCR + match confidence math
│   ├── package.json
│   └── .env.example
│
├── user-profile/                # CRUD for user dietary/allergy profiles
│   ├── src/
│   │   └── main.js
│   └── package.json
│
└── seed-database/               # One-time: populate ingredient DB from datasets
    ├── src/
    │   ├── main.js
    │   ├── seeders/
    │   │   ├── fdaFood.js
    │   │   ├── fdaCosmetics.js
    │   │   ├── euCosIng.js
    │   │   └── commonAllergens.js
    │   └── data/                  # Raw CSV/JSON source files
    └── package.json
```

---

2. Appwrite Schema Blueprint

2.1 — Database: `safescan_db`

Collection: `users_profiles`

> Stores each user's personal safety preferences — the filters applied on every scan.

Attribute	Type	Required	Description
`userId`	`string(36)`	✅	Appwrite Auth user ID (indexed, unique)
`displayName`	`string(128)`	❌	User's display name
`allergens`	`string[]`	❌	Array of allergen keys: `["peanuts","gluten","shellfish"]`
`dietaryPrefs`	`string[]`	❌	`["vegan","halal","keto"]`
`jurisdiction`	`string(8)`	✅	ISO country code → rules engine: `"US"`, `"EU"`, `"NG"`
`sensitiveIngredients`	`string[]`	❌	Custom ingredients user wants flagged
`createdAt`	`datetime`	✅	Auto-set
`updatedAt`	`datetime`	✅	Auto-set on update

Indexes: `userId` (unique), `jurisdiction`

Collection: `ingredients`

> The canonical knowledge base — every known ingredient with its regulatory and scientific profile.

Attribute	Type	Required	Description
`canonicalName`	`string(256)`	✅	Primary standardized name
`synonyms`	`string[]`	✅	All known alternative names, abbreviations, INCI names
`category`	`string(32)`	✅	`"preservative"`, `"emulsifier"`, `"allergen"`, `"colorant"`, `"humectant"`, etc.
`productTypes`	`string[]`	✅	Where it appears: `["food","skincare","haircare","makeup"]`
`function`	`string(512)`	❌	Plain-language description of what it does
`isBeneficial`	`boolean`	❌	Has recognized positive properties
`benefitDescription`	`string(1024)`	❌	Why it's considered beneficial
`riskLevel`	`string(16)`	✅	`"none"`, `"low"`, `"moderate"`, `"high"`, `"prohibited"`

Attribute	Type	Required	Description
`riskDescription`	`string(1024)`	❌	Why it's flagged — evidence summary
`isAllergen`	`boolean`	✅	Is a recognized major allergen
`allergenGroup`	`string(64)`	❌	`"tree_nuts"`, `"dairy"`, `"gluten"`, etc.
`regulatoryStatus`	`string(32)`	✅	`"approved"`, `"restricted"`, `"prohibited"`, `"gras"`, `"pending_review"`
`regulatoryNotes`	`string(2048)`	❌	Jurisdiction-specific restrictions/conditions
`jurisdictions`	`string[]`	✅	Which jurisdictions this data applies to: `["US","EU","NG"]`
`maxConcentration`	`float`	❌	Max allowed % (cosmetics) — `null` if unlimited
`sourceReferences`	`string[]`	✅	`["FDA-SAFC-2024","EU-CosIng-Annex-II"]` — audit trail
`lastVerified`	`datetime`	✅	When this record was last checked against sources

Indexes: `canonicalName` (unique), `synonyms` (fulltext), `category`, `riskLevel`, `regulatoryStatus`, `allergenGroup`

Collection: `scan_reports`

> Every scan result — the complete evidence trail from image to verdict.

Attribute	Type	Required	Description
`userId`	`string(36)`	✅	Who scanned
`imageFileId`	`string(36)`	✅	Appwrite Storage file ID of the uploaded photo
`status`	`string(16)`	✅	`"processing"`, `"completed"`, `"failed"`, `"needs_review"`
`productName`	`string(256)`	❌	Extracted product name
`productCategory`	`string(32)`	❌	`"food"`, `"beverage"`, `"skincare"`, `"haircare"`, `"makeup"`, `"soap"`, `"body_lotion"`
`rawOcrText`	`string(8192)`	❌	Full raw OCR output for audit
`ocrConfidence`	`float`	❌	0.0–1.0 — OCR quality score
`extractedIngredients`	`string[]`	❌	Raw ingredient strings as extracted
`normalizedIngredients`	`string(8192)`	❌	JSON string — array of `{raw, canonical, matchConfidence}`

Attribute	Type	Required	Description
matchConfidence	float	❌	0.0–1.0 — average ingredient match quality
overallAssessment	string(32)	❌	"generally_favorable", "use_with_caution", "high_concern", "insufficient_evidence"
benefits	string(4096)	❌	JSON string — array of {ingredient, description, evidenceLevel}
concerns	string(4096)	❌	JSON string — array of {ingredient, severity, description, source}
nutritionFlags	string(2048)	❌	JSON string — {highSodium, highSugar, highSatFat, transFat, ...}
allergenFlags	string[]	❌	Allergen groups found: ["gluten","dairy"]
userSpecificWarnings	string(2048)	❌	JSON string — warnings matched against user profile
fullReport	string(16384)	❌	JSON string — complete structured report for rendering
explanationText	string(8192)	❌	Gemini-generated plain-language summary
createdAt	datetime	✅	Scan timestamp

Indexes: userId (list), status, productCategory, overallAssessment, createdAt (descending)

Collection: ingredient_corrections

> User-submitted corrections — the feedback loop for improving accuracy.

Attribute	Type	Required	Description
reportId	string(36)	✅	Links to the scan report
userId	string(36)	✅	Who submitted the correction
originalIngredient	string(256)	✅	What the system extracted
correctedIngredient	string(256)	✅	What the user says it should be
status	string(16)	✅	"pending", "approved", "rejected"
createdAt	datetime	✅	Submission timestamp

Indexes: reportId, status

---

3. Phased Execution Plan

Phase 1 — Foundation & Skeleton (Days 1–3)

Goal: Bootable Expo app with Appwrite auth, navigation, and design system.

Step	What We Build	Files Created
1.1	Initialize Expo project with Expo Router	`app/_layout.tsx`, `app/index.tsx`
1.2	Install & configure Appwrite React Native SDK	`services/appwrite.ts`
1.3	Build the design system — theme, tokens, core UI components	`constants/theme.ts`, all `components/ui/*`
1.4	Auth flow — Welcome → Login → Register	`app/(auth)/*`, `services/auth.service.ts`, `stores/useAuthStore.ts`
1.5	Tab navigation skeleton	`app/(tabs)/*`
1.6	User profile screen + Appwrite collection	`app/(tabs)/profile.tsx`, `services/profile.service.ts`, `components/profile/*`

Terminal commands for this phase:

```
# 1. Create the Expo project
npx create-expo-app@latest mobile --template blank-typescript
cd mobile

# 2. Install core dependencies
npx expo install react-native-appwrite react-native-url-polyfill
npx expo install expo-router expo-linking expo-constants expo-status-bar
npx expo install expo-camera expo-image-picker expo-file-system
npm install zustand
npm install @expo-google-fonts/inter @expo-google-fonts/outfit

# 3. Install dev dependencies
npm install --save-dev @types/react @types/react-native
```

Phase 2 — Camera, Upload & Processing Pipeline (Days 4–6)

Goal: User can photograph a product, upload to Appwrite Storage, trigger the cloud function, and see a processing state.

Step	What We Build	Files Created
2.1	Camera capture with overlay + quality guidance	`components/scan/*`, `hooks/useCamera.ts`

Step	What We Build	Files Created
2.2	Image picker (gallery alternative)	`hooks/useImagePicker.ts`
2.3	Image compression + upload to Appwrite Storage	`services/storage.service.ts`, `utils/imageHelpers.ts`
2.4	Scan service — upload → create report doc → trigger function	`services/scan.service.ts`, `stores/useScanStore.ts`
2.5	Processing animation screen	`components/scan/ScanProgress.tsx`
2.6	Appwrite Function: `assess-product` scaffold	`functions/assess-product/src/main.js` + utils

Terminal commands for this phase:

```
# In the functions directory
mkdir -p functions/assess-product/src
cd functions/assess-product
npm init -y
npm install node-appwrite @google/genai

# Initialize Appwrite CLI (if not done)
npm install -g appwrite-cli
appwrite login
appwrite init project
appwrite init function  # Select node-20.0 runtime
```

Phase 3 — AI Engine & Safety Rules (Days 7–10)

Goal: The Appwrite function processes images end-to-end: OCR → classify → extract → normalize → safety lookup → assessment → report.

Step	What We Build	Files Created
3.1	Gemini Vision OCR integration	`functions/assess-product/src/ocr/extractText.js`
3.2	Product classification via Gemini	`functions/assess-product/src/classification/classifyProduct.js`
3.3	Ingredient extraction + normalization	`functions/assess-product/src/extraction/*`, `functions/assess-product/src/normalization/*`
3.4	Ingredient DB seeder (FDA + CosIng data)	`functions/seed-database/*`
3.5	Safety lookup against Appwrite DB	`functions/assess-product/src/safety/lookupIngredients.js`

Step	What We Build	Files Created
3.6	Deterministic food safety rules	`functions/assess-product/src/safety/foodRules.js`
3.7	Deterministic cosmetic safety rules	`functions/assess-product/src/safety/cosmeticRules.js`
3.8	Nutrition threshold rules	`functions/assess-product/src/safety/nutritionRules.js`
3.9	Score engine + overall assessment	`functions/assess-product/src/assessment/scoreEngine.js`
3.10	Gemini explanation generator	`functions/assess-product/src/assessment/explanationGenerator.js`

Terminal commands for this phase:

```
# Seed the ingredients database
cd functions/seed-database
npm init -y
npm install node-appwrite csv-parse

# Deploy functions
appwrite push function
```

Phase 4 — Report UI & History (Days 11–13)

Goal: Beautiful report rendering, scan history, product comparison, and sharing.

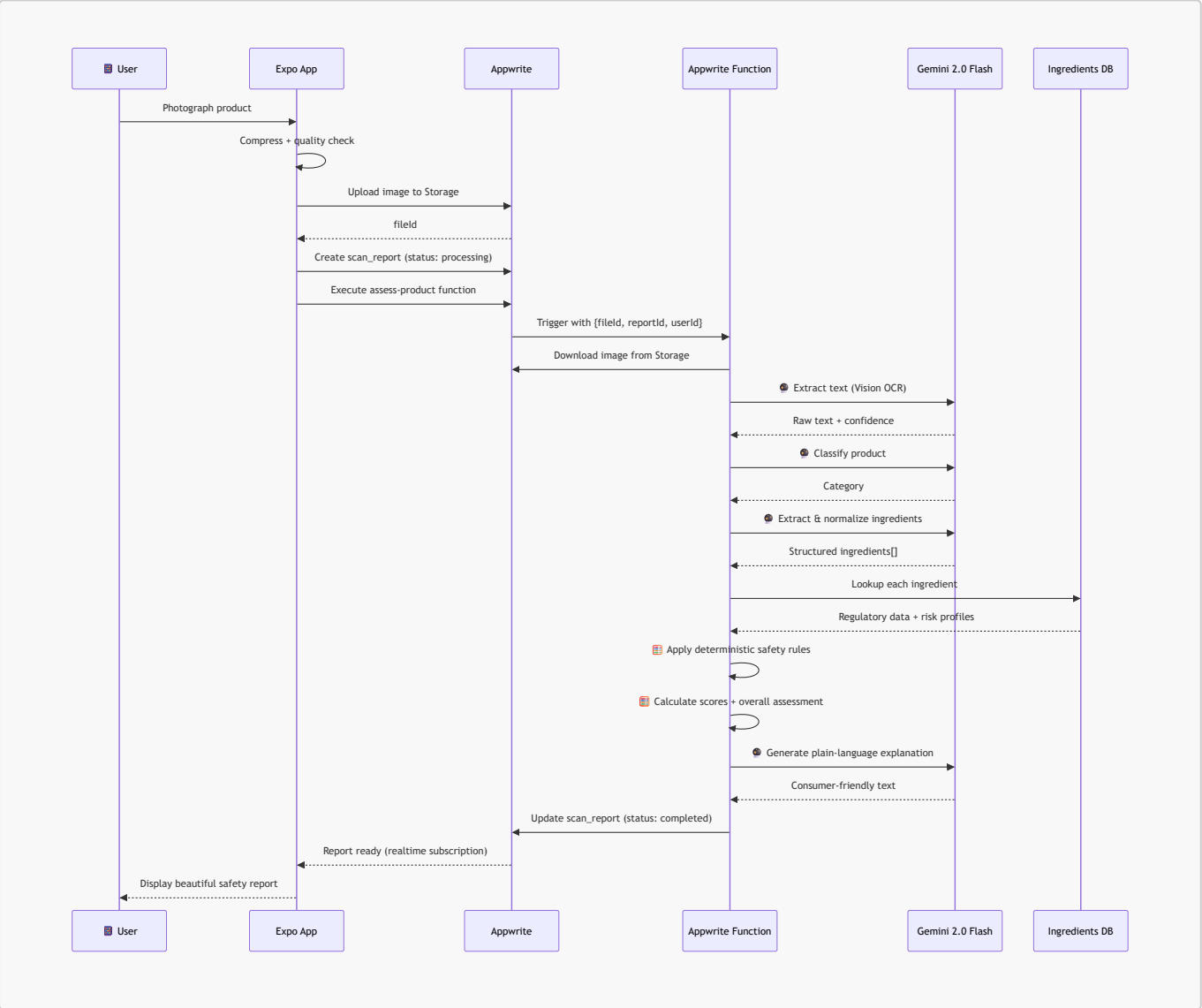
Step	What We Build	Files Created
4.1	Full report screen with all sections	`app/report/[id].tsx`, all `components/report/*`
4.2	Assessment badge system (animated)	`components/ui/Badge.tsx` (enhanced)
4.3	Confidence meter animation	`components/ui/ConfidenceMeter.tsx`
4.4	Ingredient-by-ingredient breakdown	`components/report/IngredientList.tsx`, `IngredientRow.tsx`
4.5	Scan history list	`app/(tabs)/history.tsx`, `services/report.service.ts`
4.6	Product comparison screen	`app/(tabs)/compare.tsx`
4.7	Share report as image/PDF	Utility in `utils/`
4.8	User correction submission	`components/report/CorrectionModal.tsx`

Phase 5 — Polish, Testing & Hardening (Days 14–16)

Goal: Production-quality app with error handling, offline states, accessibility, and testing.

Step	What We Build	Files Created
5.1	Error boundaries + graceful failure states	Various
5.2	Offline indicator + cached reports	Enhancement to stores
5.3	Accessibility audit — screen reader labels, contrast	All components
5.4	Image quality pre-check (blur detection, size)	`utils/imageHelpers.ts` enhanced
5.5	Rate limiting + input validation on functions	Function middleware
5.6	End-to-end testing with test product images	Test suite
5.7	Performance optimization — lazy loading, skeleton screens	Various

4. Architecture Flow Diagram



5. Security Architecture

> [!IMPORTANT] The React Native app **NEVER** calls the Gemini API directly. All AI processing happens inside Appwrite Functions where the API key is stored as an environment variable.

Layer	Security Measure
Client → Appwrite	HTTPS only, Appwrite project API key scoped to client
Auth	Appwrite Auth (email/password, OAuth) — JWT sessions
Image Upload	Appwrite Storage with per-user bucket permissions
Function Execution	Server-side only, `GEMINI_API_KEY` in env vars, never exposed
Database	Collection-level permissions: users read own reports, ingredients are public-read
Data at Rest	Appwrite encrypts storage and database
User Data	Delete account → cascade delete all images, reports, profile

---

6. First Step — Exact Terminal Commands

> [!TIP] Run these commands **in order** to initialize the project and be ready for Phase 1.

```
# ──────────────────────────────────────────────────────
# STEP 1: Navigate to workspace
# ──────────────────────────────────────────────────────
cd "d:\FOOD & COSMETIC PRODUCT SAFETY"

# ──────────────────────────────────────────────────────
# STEP 2: Create Expo project with TypeScript
# ──────────────────────────────────────────────────────
npx -y create-expo-app@latest mobile --template blank-typescript

# ──────────────────────────────────────────────────────
# STEP 3: Navigate into project
# ──────────────────────────────────────────────────────
cd mobile

# ──────────────────────────────────────────────────────
# STEP 4: Install Appwrite SDK + URL polyfill
# ──────────────────────────────────────────────────────
npx expo install react-native-appwrite react-native-url-polyfill

# ──────────────────────────────────────────────────────
# STEP 5: Install Expo Router & navigation deps
# ──────────────────────────────────────────────────────
npx expo install expo-router expo-linking expo-constants expo-status-bar

# ──────────────────────────────────────────────────────
# STEP 6: Install camera & image deps
# ──────────────────────────────────────────────────────
```

```
npx expo install expo-camera expo-image-picker expo-file-system

# ──────────────────────────────────────────────────────
# STEP 7: Install state management
# ──────────────────────────────────────────────────────
npm install zustand

# ──────────────────────────────────────────────────────
# STEP 8: Install premium fonts
# ──────────────────────────────────────────────────────
npx expo install @expo-google-fonts/inter @expo-google-fonts/outfit expo-font
expo-splash-screen

# ──────────────────────────────────────────────────────
# STEP 9: Create the functions directory structure
# ──────────────────────────────────────────────────────
cd ..
mkdir functions
mkdir functions\assess-product
mkdir functions\assess-product\src
mkdir functions\seed-database
mkdir functions\seed-database\src

# ──────────────────────────────────────────────────────
# STEP 10: Initialize the Appwrite Function package
# ──────────────────────────────────────────────────────
cd functions\assess-product
npm init -y
npm install node-appwrite @google/genai

# ──────────────────────────────────────────────────────
# STEP 11: Return to workspace root
# ──────────────────────────────────────────────────────
cd "d:\FOOD & COSMETIC PRODUCT SAFETY"
```

Decisions Made ✅

Decision	Choice
Appwrite Instance	Appwrite Cloud (`cloud.appwrite.io`)
Auth Strategy	Email/Password + Google OAuth + Apple Sign-In
AI Provider	**Multi-provider abstraction layer** — user can plug in any AI API key (Gemini, OpenAI, Claude, etc.) via a unified provider interface
Target Jurisdictions	Africa-focused: Nigeria (NAFDAC), Kenya (KEBS), South Africa (SAHPRA/NRCS), Ghana (FDA-Ghana), + US/EU as baseline references
Design Direction	Dark-mode-first, glassmorphism, emerald→teal (safe) / amber→red (concern) gradients

> [!NOTE] **AI Abstraction Layer**: The Appwrite function will use a `providers/` pattern where each AI provider (Gemini, OpenAI, etc.) implements a common interface. The active provider is selected via an environment variable (`AI_PROVIDER=gemini`) and the API key is stored separately (`AI_API_KEY=...`). This means zero code changes to switch providers.

> [!IMPORTANT] **Awaiting**: Appwrite Project ID — will be wired in when provided.